Property-based testing (PBT)

November 3rd, 2025
Ana Paiva, Auri Vincenzi, José Campos

In this recitation class, we are going to exercise ‘Property-based testing’ using the jqwik library.

⚠️ Please make sure your machine is configured properly, i.e., a bash-based shell, Java JDK 11, Apache Maven v3.9.11, and that you have the latest version of IntelliJ IDEA installed. If not, please visit the 0. Setup document.

1. Perform ‘Property-based testing’

1.0 (warm up) The `PassingGrade` class

Given the following class we saw in the lecture class, apply PBT.

```java
// A student passes an exam if they get a grade >= 9.5 and fails if they
// get a grade < 9.5.  A grade falls in the range [0.0, 20.0].
public class PassingGrade {
    public boolean passed(float grade) {
        assert grade >= 0.0 && grade <= 20.0;
        return grade >= 9.5;
    }
}
```

🧑‍🏫Tip:

1. Develop three JUnit test cases, as you have done for the last weeks, that test the following behaviours:
- one for the `pass` behaviour, i.e., any value between 9.5 (inclusive and 20.0 (inclusive).
- one for the `fail` behaviour, i.e., any value between 0.0 (inclusive) and 9.5 (exclusive).
- and one for the `invalid` behaviour, i.e., any negative or value higher than 20.0 (exclusive).

⚠️ How many inputs are you using to test the `passed` method? Just one per test case, and each one is hardcoded in the test.

2. Then, instead of having hardcoded values in your test cases, use a random generator. Here

is a quick example on how to generate a random float in Java, assuming, for example, min 0.0 and max (exclusive) 9.5:

```java
new Random().nextFloat(0.0, 9.5)
```

⚠️ How many inputs are you using to test the `passed` method? Still one per test case, right?
⚠️ Are your test cases testing the boundaries of the `passed` method? Do any of your test cases exercise the method under test with the input `20.0` ?

3. Make your `fail` test to randomly generate and use 100 floats. Tip: either implement 100 different tests, create a loop within the test, or replace the `@Test` annotation with `@RepeatedTest(100)`.

4. Inject a *bug*, e.g.,

```
-         return grade >= 9.5;
+         return grade >= 7.5;
```

5. Run your test suite, at least one test case should fail.
⚠️ Which input(s) failed?

6. Simplify your three tests with `@Property` tests.
⚠️ Which input(s) failed?

1.1 The `unique` method

Given the following method available in the Apache Commons Math project, apply PBT.

🤷 Challenge: how do you ensure that jqwik generates a list/array with repeated elements? How often are repeated elements? Note: If you do not have a list/array with repeated elements, are you really testing the `unique` function?

```java
import java.util.Iterator;
import java.util.TreeSet;
```

```java
/**
 * Returns an array consisting of the unique values in data.  The
return
 * array is sorted in descending order.  Empty arrays are allowed,
but
 * null arrays result in a NullPointerException.  Infinities are
 * allowed.
 *
 * data: Array to scan
 *
 * The method returns a descending list of values included in the
input
 * array.  It throws a NullPointerException if data is null.
 */
public static double[] unique(double[] data) {
    TreeSet<Double> values = new TreeSet<Double>();
    for (int i = 0; i < data.length; i++) {
        values.add(data[i]);
    }

    final int count = values.size();
    final double[] out = new double[count];
    Iterator<Double> iterator = values.descendingIterator();
    int i = 0;
    while (iterator.hasNext()) {
        out[i++] = iterator.next();
    }

    return out;
}
```

1.2 The `indexOf` method

Given the following method available in the Apache Commons Lang project, apply PBT.

Java

```java
/**
 * Finds the index of the given value in the array starting at the
given
 * index.  This method returns -1 for a null input array.  A
negative
 * startIndex is treated as zero.  A startIndex larger than the
array
 * length will return -1.
 *
 * array: Array to search for the object. May be null.
 * valueToFind: Value to find.
 * startIndex: Index at which to start searching.
 *
 * The method returns the index of the value within the array, or
-1 if
 * not found or null.
 */
public static int indexOf(final int[] array, final int valueToFind,
                          int startIndex) {

    if (array == null) {
        // The method accepts a null array and returns -1 in such a
        // case.  Another option could be to throw an exception or
        // having an assert.
        return -1;
    }

    if (startIndex < 0) {
        // The method accepts a negative startIndex.  In such case,
it
        // assumes it is 0.  Another option could be to throw an
        // exception or having an assert.
        startIndex = 0;
    }

    for (int i = startIndex; i < array.length; i++) {
```

```
        if (valueToFind == array[i]) {
            return i;
        }
    }

    return -1;
}
```

You can find below the manually-written test case developed by Lang's developers for the `indexOf` method.
(🧑‍💻 extra homework exercises: (1) does the manually-written test case exercise all partitions? (2) convert the manually-written test case into a parameterized version of it.)

Java
```
@Test
public void testIndexOfIntWithStartIndex() {
    int[] array = null;
    assertEquals(-1, indexOf(array, 0, 2));
    array = new int[]{0, 1, 2, 3, 0};
    assertEquals(4, indexOf(array, 0, 2));
    assertEquals(-1, indexOf(array, 1, 2));
    assertEquals(2, indexOf(array, 2, 2));
    assertEquals(3, indexOf(array, 3, 2));
    assertEquals(3, indexOf(array, 3, -1));
    assertEquals(-1, indexOf(array, 99, 0));
    assertEquals(-1, indexOf(array, 0, 6));
}
```

🧑‍💻Tip to apply PBT to the `indexOf` method: the overall idea of the test is to insert a random value (i.e., an integer value) in a random position of a random array of integers. The `indexOf` method will then look for this random value. Finally, the test will `assert` that the method returns an index that matches the random position where we inserted the element.

🧑‍💻Tip: in case you plan to generate random lists of integers with jqwik, please find below a utility method that converts a list of integers to an array (which is required by `indexOf`).

Java

```java
private int[] convertListToArray(List<Integer> numbers) {
    int[] array = numbers.stream().mapToInt(x -> x).toArray();
    return array;
}
```

1.3 MyStringStack

Given the following implementation of a stack that only handles strings, apply PBT.

Java

```java
import java.util.ArrayList;
import java.util.List;

public class MyStringStack {

	private final List<String> elements = new ArrayList<String>();

	public void push(final String element) {
		this.elements.add(0, element);
	}

	public String pop() {
		return this.elements.remove(0);
	}

	public void clear() {
		this.elements.clear();
	}

	public boolean isEmpty() {
		return this.elements.isEmpty();
	}

	public int size() {
		return this.elements.size();
	}
```

```
    public String top() {
        return this.elements.get(0);
    }

    @Override
    public String toString() {
        return this.elements.toString();
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (o == null || this.getClass() != o.getClass()) {
            return false;
        }

        MyStringStack that = (MyStringStack) o;
        return this.elements.equals(that.elements);
    }

    @Override
    public int hashCode() {
        return this.elements.hashCode();
    }
}
```

As any object in with state, a stack can be considered to be a [finite state machine] which comes with a few characteristics:

1. It always has a defined current state – out of a finite list of possible states.
2. For every state there is a finite and possible empty list of transitions that bring the state machine into its next state.
3. Transitions are often triggered by actions (aka events).

One way to represent a state machine is a state transition table. Here's one for MyStringStack:

Current State	Action	Next State
empty	push	filled
empty	clear	empty
filled	push	filled
filled (with just one element)	pop	empty
filled (with more than one)	pop	filled
filled	clear	empty

This table reveals a few things:
- It can give us a hint about what transitions we might want to test.
- Not all actions are allowed in all states, e.g. `pop` does not make sense for an empty stack.
- A stack has additional and important functional properties that are not visible in the transition table, e.g. `pop` should always return the last element that was `push`ed on the stack. Those functional properties can typically be considered as *postconditions*.

Looking at this from a property-based testing point of view the following approach seems viable:
1. Generate a random sequence of actions.
2. Apply this sequence to a state machine in its initial state.
3. For any (allowed) action, check that the resulting state is correct and that any other invariants and postconditions hold.

🧑‍🏫Tip: check how *actions* and property-based tests that generate *actions* were implemented in the lecture class.

Inject the following *bug*:

```
None
-         elements.clear();
+         if (elements.size() < 4) elements.clear();
```

⚠️ Does your test suite detect this bug?

1.4 Back to the Pacman game

Given the tests defined in the 3. Equivalent Class Partitioning; Boundary Value Analysis (s… recitation class, in particular the ones for the methods `squareAt` and `createBoard`, convert all non-property-based test cases to `@Property`.

Miscellaneous

- JUnit framework
- JUnit 5 User Guide
- Guide to Configuring Maven Plug-ins
- jqwik
- jqwik's user guide v1.9.3